



International University of Sarajevo

Faculty of Engineering and Natural Sciences

Operating Systems

CS307 FALL 2025

PROJECT 1 REPORT

Thread Management and Scheduling in Embedded Systems

Professor: Ali Abd Almisreb

Students: Hena Šehović

Berina Juković

Berin Žunić

Sarajevo, 09.11.2025.

Contents

1. Introduction.....	1
2. Process of Making Thread Management and Scheduling in Embedded Systems	2
2.1 System Overview	2
2.2 Hardware configuration.....	3
2.3 Software components	5
2.4 Task description.....	7
2.5 RTOS communication and scheduling	10
3. HTML Simulation.....	12
4. Conclusion	14
5. References.....	15
6. List of Figures	15

1. Introduction

Computing technology has brought with it a world with intelligence literally embedded into our everyday living. From control units in cars to medical and smart home devices, embedded systems are almost everywhere. One thing that basically characterizes these systems is that they function in highly constrained conditions. In other words, embedded systems are designed to operate with very limited processing power and small amounts of memory, yet they are often required to run with high efficiency and low energy consumption.

Meeting these demands using a single processor makes multitasking extremely important. In fact, multitasking is one of the key topics explored in this project. The focus is on understanding how efficient multitasking can be achieved in systems with such strict hardware constraints. The hardware part of this project was implemented on an STM32 microcontroller, and a complimentary simulation was built using HTML, CSS, and JavaScript for better visualisation of the behaviour and scheduling decisions.

To achieve multitasking in embedded systems, one of the most efficient ways is using threads. Threads are lightweight units of execution within a process that share the same address space and resources, allowing multiple tasks to run seemingly in parallel. This shared environment makes threads highly effective for embedded systems where memory and processing power are limited.

Each thread has its own stack, program counter, and set of registers, which store its current state and allow it to run independently. The scheduler or real-time operating system (RTOS) keeps important information about every thread in a data structure called the Thread Control Block (TCB). When the system switches from one thread to another (context switch) it saves the current thread's state and loads the next thread's state from its TCB.

Scheduling decisions are usually triggered by timer interrupts, input/output (I/O) events, or when a thread voluntarily yields control of the processor. To make sure multiple threads can safely share resources, embedded systems use synchronization mechanisms such as mutexes, semaphores, message queues, and event flags. These tools help prevent conflicts and ensure threads work together smoothly without interfering with each other's data.

Early embedded systems were usually single-threaded or used very simple cooperative schedulers because hardware resources were very limited. An introduction of RTOSes and microcontrollers with more capabilities enabled wide usage of multithreading in embedded systems.

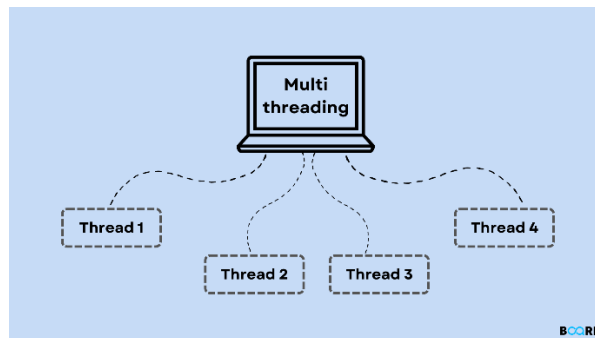


Figure 1: Visual Representation of Multithreading

2. Process of Making Thread Management and Scheduling in Embedded Systems

2.1 System Overview

The embedded operating system to be developed in this project is a multitasking application running on the STM32 microcontroller using the CMSIS-RTOS2 FreeRTOS real-time operating system. In this setup, the main objective will be to illustrate how several threads can run concurrently, share information using inter-task communication mechanisms, and manage hardware peripherals, including GPIO and UART, in an organized and predictable way.

The main role of the system is the scheduler of RTOS, which takes all active threads controlled into account and distributes CPU time to them according to their priority level as well as the status of the whole system. Six pthreads are generated: one built-in idle task, three LED control tasks, one uart related task and once calculation process. 50 These threads are executed individually and carry out a particular function, thereby achieving the real parallel execution time at the software level. This illustrates one of the key aspects of an RTOS, concurrent tasks and cooperative multitasking.

These three LED tasks- ledTask1, ledTask2, and ledTask3-toggle different GPIO pins, to which LEDs are connected, at regular intervals: PC4, PC5, and PC6. These tasks illustrate how independent processes can operate simultaneously under the control of the RTOS scheduler. By observing the LEDs blinking concurrently, one can visually confirm that multiple tasks are being managed in parallel rather than sequentially.

The computation exercise is intended to model a process that generates data. It sends an incremented atomic variable to a message queue at intervals. This is a good example of one the most important concepts in RTOS systems, which is inter-task communication: tasks use thread-safe queues to communicate, and this prevents data corruption when multiple threads operate concurrently.

Complementing this, the UART message task could be regarded as a consumer of data: it waits for a thread flag from the calculation task, fetches the latest counter value from the message queue, and sends it via the USART2 interface to a serial terminal. This setup ensures that a UART transmission only occurs when valid data is available, with no superfluous polling or conflicts over resources. The use of thread flags allows event-driven synchronization between the producer-the calculation task-and the consumer-the UART task-efficiently and scalably.

In addition to this task-level structure, the system incorporates a variety of low-level peripherals that are configured using STM32CubeMX. The GPIO subsystem provides the driver for the LED output operations, and the USART2 peripheral is configured for serial communication with the PC terminal at a baud rate of 115200. The system clock is generated using the internal HSI oscillator with a PLL configuration, thus providing a stable system frequency for reliable task scheduling and accuracy in timing.

General, there is the good embedded system example in which each functionality can be a separate RTOS thread. The integration of task scheduling, message exchange and hardware control indicate that FreeRTOS has a promising prospect in embedded field due to its attractive features including modularity, reliability and availability. This design reflects the real-world

embedded systems, in which M tasks (e.g., sensor reading, data processing and communication) run concurrently in precise timing required.

2.2 Hardware configuration

The hardware platform used in this project will be the STM32 microcontroller, which is a part of the STMicroelectronics ARM Cortex-M family, which is also known for its low power consumption and efficient processing, coupled with broad peripheral support. A specific development board, for instance, the Nucleo-F401RE or STM32F4 Discovery, has an onboard STM32 MCU, USB for programming, a user LED, and onboard GPIO headers, making it ideal for prototyping real-time operating system applications.

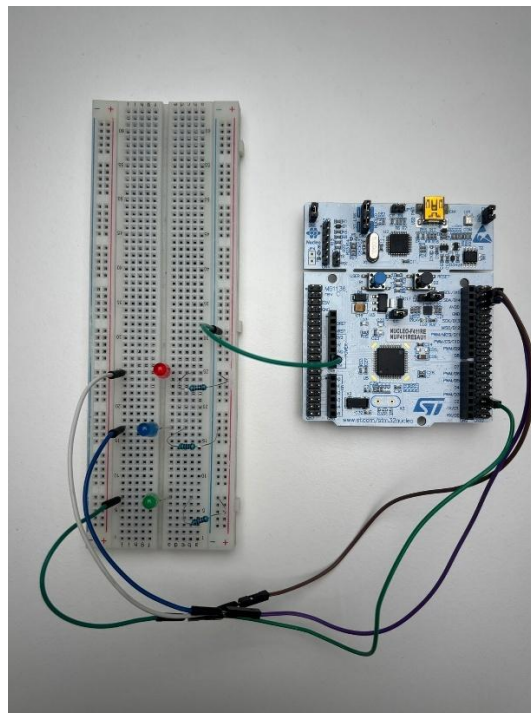


Figure 2: Hardware Setup

For this project, several key peripherals are configured to support multitasking operations and peripheral interactions. Firstly, General Purpose Input/Output (GPIO) is used mainly for LED control. Three output pins of Port C, namely PC4, PC5, and PC6, are configured as digital outputs to drive external LEDs. Each pin will be toggled by an RTOS thread independently of the other. This will provide an excellent visual indication of the execution of tasks running in parallel. LEDs will blink simultaneously at fixed intervals as a representation of concurrent scheduling of threads by the FreeRTOS kernel. The GPIO mode is configured as push-pull with no internal pull-up or pull-down resistors, while the output speed is set to low frequency for power and signal noise minimization.

The USART2 peripheral is configured for serial communication between the STM32 and a computer. It provides a way for the system to send runtime information - such as counter values produced by the calculation task, to a terminal program, such as Tera Term or PuTTY, for display. The UART is configured for 115200 baud rate, 8 data bits, 1 stop bit, no parity, and standard TX/RX interface. The UART is initialized through `MX_USART2_UART_Init()` to

ensure that the hardware and the HAL layer are correctly set up before the RTOS starts scheduling.

Deterministic task timing and stable peripheral operation is achieved with a properly configured system clock. The High-Speed Internal (HSI) oscillator will be used as the input source to drive the PLL to generate the higher system frequency. This kind of configuration would keep the microcontroller at an optimal speed both for the RTOS kernel and peripheral timing requirements. Clocks are branched to the AHB and APB buses with separate clocking domains: core, peripherals, and communications.

To ensure consistent system operation, the power control and flash latency settings are also configured during clock setup. The voltage regulator is set to Scale 1 mode for full performance operation, and the flash latency is adjusted according to the CPU frequency to maintain reliable instruction fetching. During clock setup, power control and flash latency settings are also configured for consistent system operation. The voltage regulator is set to Scale 1 mode for full performance operation; the flash latency is adjusted according to the CPU frequency for the purpose of maintaining reliable instruction fetching.

Overall, the hardware configuration is a stable and flexible backbone for multitasking in FreeRTOS. Each of the integrated hardware peripherals, such as GPIO for LED output, USART2 for communication, and system clock for synchronization with the RTOS layer, ensures that the task scheduling is smooth, timing control accurate, and data exchange reliable characteristics all real-time embedded systems should have.

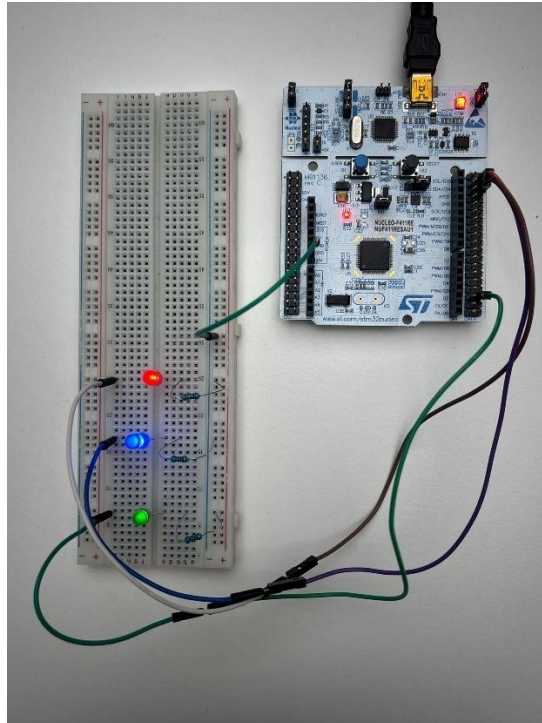


Figure 3: Hardware with the Code Running

2.3 Software components

This project's software is being developed with STM32CubeIDE, which integrates code generation from the STM32CubeMX tool and is a complete development environment for STM32 microcontrollers. The firmware is based on the HAL and CMSIS-RTOS2 frameworks. These layers make peripheral configuration easier and the code portable to most of the STM32 families while providing real-time operating system services like task scheduling, message queues, and inter-task synchronization.

The main() performs all the necessary hardware initializations and prepares the RTOS environment during system startup. Once the STM32 HAL library is initialized with a call to HAL_Init(), the system clock is configured by calling the function SystemClock_Config(), which initializes both the internal HSI oscillator and PLL to provide the microcontroller with the proper operating frequency. The next steps involve initialization of the peripheral drivers by calling MX functions, including the initialization of the LED output pins by the MX_GPIO_Init() and serial communication interface MX_USART2_UART_Init(). These are automatically generated in STM32CubeMX, which ensures that before the start of application tasks, all hardware resources are configured.

```
osThreadId_t ledTask1Handle;
const osThreadAttr_t ledTask1_attributes = {
    .name = "ledTask1",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityLow,
};
/* Definitions for ledTask2 */
osThreadId_t ledTask2Handle;
const osThreadAttr_t ledTask2_attributes = {
    .name = "ledTask2",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityLow,
};
/* Definitions for ledTask3 */
osThreadId_t ledTask3Handle;
const osThreadAttr_t ledTask3_attributes = {
    .name = "ledTask3",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityLow,
};
/* Definitions for msguart */
osThreadId_t msguartHandle;
const osThreadAttr_t msguart_attributes = {
    .name = "msguart",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityLow,
};
/* Definitions for calculation */
osThreadId_t calculationHandle;
const osThreadAttr_t calculation_attributes = {
    .name = "calculation",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityLow,
};
/* Definitions for calculation_msg */
osMessageQueueId_t calculation_msgHandle;
const osMessageQueueAttr_t calculation_msg_attributes = {
    .name = "calculation_msg"
};
```

Figure 4: The Code segment defining threads attributes and messaging queue configuration for the STM32 FreeRTOS application

Once the hardware setup is complete, the program initializes the RTOS kernel by making a call to `osKernelInitialize()`. A multitasking system is then defined by the creation of several threads, a message queue, and thread attributes that define name, priority, and stack size. Each of the threads is implemented in its own function, which executes independently under the control of the FreeRTOS scheduler. The three LED tasks called `ledTask1`, `ledTask2`, and `ledTask3` toggle different GPIO pins at fixed intervals using the `HAL_GPIO_TogglePin()` function combined with an `osDelay()` function call for creating a half-second delay. In this case, concurrent execution of multiple independent processes on a single CPU implies time-slicing.

This calculation task is a data producer: it periodically increases the counter variable, putting the updated value into the message queue with the help of the `osMessageQueuePut()` function. After updating the queue, this task performs `osThreadFlagsSet()` to notify the msguart task-a data consumer. A UART task waits for an event using the `osThreadFlagsWait()` function, takes the latest message from the queue, and sends it through the serial port using the `HAL_UART_Transmit()`. This kind of event-driven communication allows the UART task to run only when new data arrives, which saves CPU overhead and demonstrates a good pattern of synchronization in RTOS applications.

```
osKernelInitialize();

/* Create the queue(s) */
/* creation of calculation_msg */
calculation_msgHandle = osMessageQueueNew(16, sizeof(uint16_t), &calculation_msg_attributes);

defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

/* creation of ledTask1 */
ledTask1Handle = osThreadNew(ledTasks1, NULL, &ledTask1_attributes);

/* creation of ledTask2 */
ledTask2Handle = osThreadNew(ledTasks2, NULL, &ledTask2_attributes);

/* creation of ledTask3 */
ledTask3Handle = osThreadNew(ledTasks3, NULL, &ledTask3_attributes);

/* creation of msguart */
msguartHandle = osThreadNew(msgsuarts, NULL, &msguart_attributes);

/* creation of calculation */
calculationHandle = osThreadNew(calucalations, NULL, &calculation_attributes);

osKernelStart();
```

Figure 5: Code segment showing the creation of all tasks and the start of the FreeRTOS Scheduler

The default task, created automatically from the CubeIDE project template, is a continuously running background task with a minimum priority. It could be used for idle operations or system extensions in future, but would not be used for this project. Finally, when all the threads and queues are created, the call `osKernelStart()` is used to transfer the control from the main function to the RTOS kernel. From this moment on, all the switching among the tasks, as well as delays or message queue operations, is handled by the FreeRTOS scheduler without any direct action of the main loop.

This is a layered software design: hardware abstraction, initialization of peripherals, RTOS kernel management, and application logic based on tasks, which ensures that the program is modular, maintainable, and responsive. Each functional block operates independently while interacting through well-defined synchronization mechanisms, characteristically in a robust real-time embedded system.

2.4 Task description

Through the use of six concurrent threads, the implemented system is intended to illustrate fundamental concepts of real-time operating systems. Together, these threads demonstrate direct hardware control, inter-task communication, and preemptive scheduling in an embedded environment with limited resources. Each task's function and behavior are described in depth in the following:

1. Idle task

The Idle Task is a built-in, low-priority thread that executes when no other tasks are ready. Its primary role is to guarantee efficient CPU usage by providing a fallback process for the scheduler, which frequently places the system into a safe, low-power state. It is an important component for maintaining system stability and power efficiency within the RTOS, even though it performs no application-level functions and does not interact with other threads.

In this project, the Idle Task is implemented as `defaultTask`, created using the `osThreadNew()` function (Figure 6) and running the `StartDefaultTask()` function (Figure 7). Its code consists of an infinite loop with a small delay (`osDelay(1)`), which ensures that it consumes minimal CPU resources. This task executes only when no other tasks are ready, allowing the FreeRTOS scheduler to maintain continuous task management.

```
/* creation of defaultTask */  
defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
```

Figure 6: Creation of the Idle Task

```
void StartDefaultTask(void *argument)  
{  
    /* USER CODE BEGIN 5 */  
    /* Infinite loop */  
    for(;;)  
    {  
        osDelay(1);  
    }  
    /* USER CODE END 5 */  
}
```

Figure 7: Implementation of the Idle Task

2. LED control tasks

Low-priority threads called LED Control Tasks are used to show how several tasks can be executed simultaneously under the FreeRTOS scheduler. Their main function is to provide a distinct, visible signal that multitasking is taking place by toggling LEDs attached to particular GPIO pins at predetermined intervals. As a straightforward yet efficient way to demonstrate real-time task management and

parallel execution in an embedded system, these jobs function independently, enabling the scheduler to distribute CPU time effectively and without interference.

In this project, the LED Control Tasks are implemented as ledTask1, ledTask2, and ledTask3, each created using the osThreadNew() function (Figure 8) and running the functions ledTasks1(), ledTasks2(), and ledTasks3() respectively as shown on the example of the first one in Figure 9. Each function consists of an infinite loop that toggles a specific GPIO pin (PC4, PC5, and PC6) using HAL_GPIO_TogglePin() and then pauses execution for 500 milliseconds with osDelay(500). The RTOS's predictable scheduling and real-time concurrency are demonstrated by the blinking LEDs, which verify that several jobs are being managed concurrently.

```
/* creation of ledTask1 */
ledTask1Handle = osThreadNew(ledTasks1, NULL, &ledTask1_attributes);

/* creation of ledTask2 */
ledTask2Handle = osThreadNew(ledTasks2, NULL, &ledTask2_attributes);

/* creation of ledTask3 */
ledTask3Handle = osThreadNew(ledTasks3, NULL, &ledTask3_attributes);
```

Figure 8: Task Creation in main()

```
void ledTasks1(void *argument)
{
    /* USER CODE BEGIN ledTasks1 */
    /* Infinite loop */
    for(;;)
    {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_4);
        osDelay(500);
    }
    /* USER CODE END ledTasks1 */
}
```

Figure 9: Implementation of an LED Control Task (ledTasks1)

3. Calculation task

As a low-priority thread, the calculation task simulates a periodic computational activity and serves as the system's data generator. Its primary purpose is to demonstrate secure inter-task communication by incrementing a counter and adding the value to a message queue. To synchronize with the consumer, it uses a thread flag to signal the UART task upon new data availability, embodying an event-driven, producer-consumer pattern.

In this implementation, the calculations() function, created as shown in Figure 10, contains an infinite loop where a counter is incremented by two. This value is then passed to the message queue via osMessageQueuePut(), see Figure 11, followed by

setting the UART_TASK_FLAG. A final osDelay(1000) call ensures the task cycles once per second, providing a clear example of deterministic, periodic execution and safe data sharing within FreeRTOS.

```
/* creation of calculation */
calculationHandle = osThreadNew(calucalations, NULL, &calculation_attributes);
```

Figure 10: Creating the Calculation Task

```
void calucalations(void *argument)
{
    /* USER CODE BEGIN calucalations */
    uint16_t counter = 0;
    for(;;){
        counter += 2;

        osStatus_t status = osMessageQueuePut(calculation_msgHandle, &counter, 0U, 0U);

        osThreadFlagsSet(msguartHandle, UART_TASK_FLAG);

        osDelay(1000);
    }
    /* USER CODE END calucalations */
}
```

Figure 11: Project's Calculation Task calculations()

4. UART communication task

The UART Communication task is a low-priority thread that shows consumer behavior in a producer-consumer pattern by sending data produced by the calculating task to a PC terminal. Its main function is to retrieve the most recent counter value from a message queue, send it over the USART2 interface, and receive notifications from the calculating job via thread flags.

In this project, the UART Communication Task is implemented as msguart, created using the osThreadNew() function (Figure 12) and running the msgsuarts() function (Figure 13). The task function consists of an infinite loop that waits for a thread flag (UART_TASK_FLAG) using osThreadFlagsWait(). Once the flag is received, it retrieves the latest value from the calculation message queue with osMessageQueueGet() and sends it to the serial terminal using HAL_UART_Transmit(). This setup allows the UART task to operate efficiently and synchronously with the calculation task.

```
/* creation of msguart */
msguartHandle = osThreadNew(msgsuarts, NULL, &msguart_attributes);
```

Figure 12: Creation of the UART Communication Task (msguart) in FreeRTOS

```

void msgsuarts(void *argument)
{
    /* USER CODE BEGIN msgsuarts */
    uint16_t value;
    char line[32];

    for(;;){
        osThreadFlagsWait(UART_TASK_FLAG, osFlagsWaitAny, osWaitForever);

        if (osMessageQueueGet(calculation_msgHandle, &value, NULL, 0) == osOK){
            int n = snprintf(line, sizeof(line), "val=%u\r\n", value);
            HAL_UART_Transmit(&huart2, (uint8_t*)line, n, HAL_MAX_DELAY);
        }
    }
    /* USER CODE END msgsuarts */
}

```

Figure 13: Implementation of the UART Communication Task Function (msgsuarts())

Preemptive multitasking is made possible while maintaining predictable behavior in a real-time system via the FreeRTOS scheduler, which controls task execution based on readiness and priority. The LED, computation, and UART tasks can all run simultaneously without interference thanks to synchronization techniques like message queues and thread flags, which offer safe and coordinated data interchange across processes.

Every thread in the system has a distinct function: the computation and UART tasks show event-driven communication, the LED tasks show parallel processing, and the idle task controls CPU utilization. Together, these activities provide a useful example of FreeRTOS capabilities on an STM32 microcontroller and highlight the fundamental concepts of thread management, real-time scheduling, and multitasking in embedded systems.

2.5 RTOS communication and scheduling

In this project, the FreeRTOS kernel is used at the core to manage how multiple tasks run concurrently on the STM32 microcontroller. The RTOS allows for real-time responsiveness, which provides deterministic scheduling of tasks and efficient mechanisms for their communication, which has a result of ensuring predictability in the behavior under multitasking conditions. Each block, be it the blinking tasks of LEDs, the UART communication task, or the calculation task within this system, can be built as a separate thread that is to be managed by the scheduler of the FreeRTOS.

In this example, several independent tasks, such as LED blinking, UART transmission, and counter incrementation, are run by FreeRTOS on STM32, all managed by the real-time kernel scheduler. Each of these tasks executes for a while and then voluntarily yields control to the scheduler using the `osDelay()` function. For example, some tasks call `osDelay(500)`, some `osDelay(1000)`, while others call `osDelay(2000)`, depending on their requirement. When a task calls `osDelay()`, it moves into the Blocked state for the duration specified by the parameter of `osDelay()` in system ticks. While this is happening, the scheduler automatically selects another Ready task to execute and automatically switches context to the selected task, thereby optimizing CPU time utilization.

FreeRTOS is based on a priority-based preemptive scheduling algorithm; that is, at any moment, it selects the highest-priority task in the Ready state to execute. The system timer generates periodic interrupts, usually every 1 millisecond, which is the heartbeat of the

scheduler. On each tick, the kernel updates the state of all tasks and decides whether a context switch is needed. If a higher-priority task becomes ready, a context switch is triggered via the PendSV interrupt, and the CPU immediately starts executing the more important task. This is transparent to the user but essential for real-time performance.

Communication between tasks in this project is achieved through message queues and thread flags that provide safe, efficient methods for data exchange and synchronization. The calculation task works as a data producer, at regular intervals inserting new values of the counter into the message queue. The UART task will wait on an event (thread flag) that signals the arrival of new data, read the message from the queue, and send it over the UART interface. An event-driven communication model based on the described principle avoids busy-waiting; tasks execute only when needed, which optimizes CPU utilization and saves energy.

As a result, all tasks run smoothly and in a deterministic manner, with each one receiving its fair share of CPU time according to its priority. Tasks of equal priority, like the three threads controlling the LEDs, share runtime in a round-robin basis, thus creating the effect of true parallel operation even though only one task executes at any instant. FreeRTOS allows the STM32 microcontroller to demonstrate genuine multitasking behavior, whereby timing precision, task priorities, and inter-task communication are all handled by the RTOS kernel to meet real-time, reliable performance of the system.

3. HTML Simulation

An interactive simulation done using HTML, CSS, and JavaScript, was made in addition to the hardware implementation. The simulation visually shows how a real time scheduler, like the one used in FreeRTOS, manages multiple threads in two different modes: sequential and parallel (Round Robin). It gives users the opportunity to see how CPU time is distributed between the threads, and how the system behaves when tasks run concurrently.



Figure 14: HTML Simulator Interface

The interface of the simulator (Figure 14) has controls for choosing the scheduling mode, quantum length, and the speed of the simulation. In the time display the execution of the three threads (T1, T2, T3) is visualized, along with the activity of the CPU, with active execution, I/O blocking, and idle periods each having a unique color indicator. In sequential mode, only one thread progresses at a time, simulating non-preemptive scheduling behaviour. In parallel mode, multiple threads make progress at the same time within a round-robin scheme.

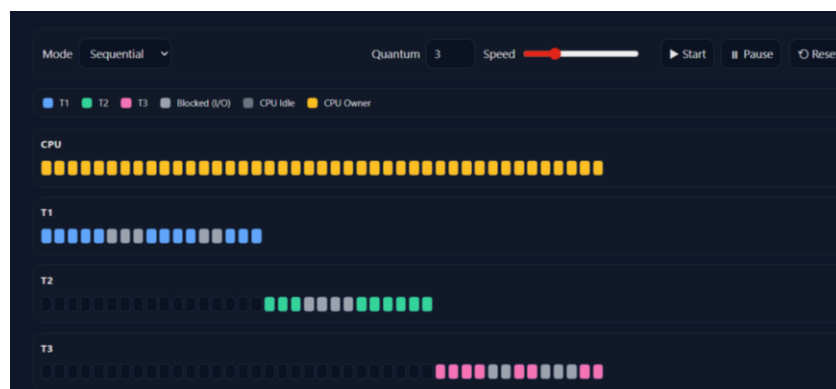


Figure 15: Vizualization of the Sequential Scheduling



Figure 16: Vizualization of the Parallel (Round Robin) Scheduling

Each tick of the simulation is a representation of a discrete unit of the CPU time. Task state transitions, queue management, and synchronization is handled by JavaScript, providing a simple model of thread scheduling that shows how the FreeRTOS system that is implemented on the STM32 works. The HTML defines the user interface, while CSS provides the appropriate design.

The simulation was done with a goal in mind – visually help understanding the connection between software thread scheduling and hardware multitasking.

4. Conclusion

Through this project we have explored the principles of thread management and real-time scheduling in embedded systems through both a practical STM32 FreeRTOS implementation and an accompanying HTML-based simulator. Together, these two platforms demonstrated how well-structured multitasking improves system responsiveness, modularity, and efficiency in resource-constrained environments. By combining theoretical scheduling concepts with practical experimentation, the work provided a deeper understanding of how scheduling strategies, and synchronization methods influence the predictability and reliability of real-time embedded applications.

The STM32 implementation successfully showcased multitasking and communication between multiple concurrent threads – shown on the example of LED blinking, UART transmission, and periodic counter updates - all coordinated by the FreeRTOS kernel. The use of message queues and thread flags enabled smooth data transfer and event-driven synchronization between producer and consumer tasks. Observations confirmed that the priority-based preemptive scheduling algorithm of FreeRTOS allows critical tasks to execute definitely, while equal-priority threads share CPU time in a round-robin manner. These behaviors were consistent with results from the HTML simulator, which served as an effective visualization and validation tool for thread timing and task scheduling.

Throughout the project, emphasis was placed on synchronization, resource management, and reliability. Techniques such as conservative stack sizing, static allocation, and minimal interrupt routines ensured predictable behavior and efficient CPU utilization. The use of RTOS synchronization primitives — including message queues, semaphores, and priority inheritance — minimized race conditions and avoided priority inversion, leading to a more stable and responsive multitasking environment. This reinforced the importance of careful system design and proper use of real-time constructs in embedded software development.

Ultimately, the project confirmed that disciplined thread design and appropriate scheduling policies are essential to achieving reliable multitasking on embedded microcontrollers. The STM32 FreeRTOS implementation, supported by the HTML simulator, provided both a practical demonstration platform and an analytical tool for studying scheduler dynamics. The findings highlight that, with structured task separation, efficient synchronization, and well-chosen priorities, embedded systems can achieve real-time performance, low power consumption, and predictable timing. These results form a strong foundation for future extensions such as adaptive scheduling, deeper power optimization, and deployment in more complex real-time applications.

5. References

FreeRTOS Download <https://www.freertos.org/>

STM32 Cube IDE Download <https://www.st.com/en/development-tools/stm32cubeide.html>

OpenAI. (2025). ChatGPT Conversations on SE-ResNet. Retrieved from <https://chatgpt.com/>

Multithreading in Embedded Systems: Design, Efficiency, and Best Practices (2023), M. Kalayci
<https://medium.com/@mkklyci/multithreading-in-embedded-systems-design-efficiency-and-best-practices-b803db0b6d15>

Threads in Distributed Systems, GeeksforGeeks (2025)

<https://www.geeksforgeeks.org/system-design/threads-in-distributed-systems/>

Synchronizing Threads in Embedded RTOS, Number Analytics (2025)

<https://www.numberanalytics.com/blog/synchronizing-threads-embedded-rtos>

Figure 1 - <https://www.boardinfinity.com/blog/content/images/2023/02/Multithreading-in-Python.png>

6. List of Figures

Figure 1: Visual Representation of Multithreading.....	1
Figure 2: Hardware Setup	3
Figure 3: Hardware with the Code Running	4
Figure 4: The Code segment defining threads attributes and messaging queue configuration for the STM32 FreeRTOS application.....	5
Figure 5:Code segment showing the creation of all tasks and the start of the FreeRTOS Scheduler	6
Figure 6: Creation of the Idle Task	7
Figure 7: Implementation of the Idle Task.....	7
Figure 8: Task Creation in main()	8
Figure 9: Implementation of an LED Control Task (ledTasks1).....	8
Figure 10: Creating the Calculation Task.....	9
Figure 11: Project's Calculation Task calculations()	9
Figure 12: Creation of the UART Communication Task (msguart) in FreeRTOS	9
Figure 13: Implementation of the UART Communication Task Function (msgsuarts())	10
Figure 14: HTML Simulator Interface	12
Figure 15: Vizualization of the Sequential Scheduling	12
Figure 16: Vizualization of the Parallel (Round Robin) Scheduling.....	13